Blockchain-Based Digital Evidence Chain of Custody

Introduction

Blockchain-based chain of custody (B-COC) is a relatively new technology that integrates digital evidence handling into a secure, immutable blockchain ledger, thus creating a tamper-resistant record of the chain of custody. As devices become increasingly interconnected, it is important for law enforcement and digital forensic analysts to preserve digital evidence. Keeping a record of evidence handling and showing that the evidence hash remains unaltered are crucial for maintaining evidence integrity. Without proper evidence handling and a record of it, evidence can be dismissed. Implementing a blockchain layer into the backend of forensic software and digital evidence storage streamlines this process.

Motivation

This project serves as a proof of concept for creating a Python-based blockchain that stores metadata related to the evidence, personnel, and actions taken with evidence. It demonstrates that blockchain concepts can be applied to digital forensics without relying on existing blockchain infrastructure. Using Python to build a blockchain from scratch enabled a thorough understanding of blockchain architecture and the cryptographic processes that maintain record integrity.

Problem Statement

While current implementations are sufficient for the chain of custody, Blockchain offers unique features such as integrity verification, non-repudiative record creation, and an immutable history of interactions with digital evidence. This project aims to explore how these features can be incorporated into digital forensics evidence management as a proof of concept.

Application Analysis

Starting with the front-end (Fig. 3), the application offers the option to sign in as an already authorized user or to register by entering a user_id. It utilizes a JSON with structured dictionary data to populate an initial ledger. From there, the user can view

chain-of-custody records for evidence they've handled and verify the entire ledger independently.

They can also submit a new block by submitting the evidence_ID and the action taken, such as analysis, transfer, collection, or storage. The list was shortened for simplicity. It also uses simple regex to ensure inputs are in valid formats and within approved character lengths. The transfer action is unique in that the user must choose a recipient from an authorized user list, simulating a secure transfer.

The user will also have to enter the hash of the digital evidence, which must match. Adding a block manually is used to simulate what forensics software would do automatically if blockchain technology were built into the backend. It also serves to add evidence-handling interactions outside of automated record-keeping.

On the back end, the block structure (Fig. 1) consists of metadata such as:

- Evidence ID
- Evidence hash
- Signer or device ID
- Source type
- Action performed
- Timestamp
- Digital signature
- Public key
- Previous hash
- Block hash
- Block ID
- Global ID

The block hash is calculated using SHA256 on a byte stream created from the global ID, block ID, evidence ID, signer ID, source type, evidence hash, timestamp, previous hash, and action taken. Any alteration of any of the header fields will result in a different hash, revealing any tampering during validation.

The blockchain structure is a list of blocks, and the class is used to validate individual blockchains in the EvidenceLedger. The blockchain validation function compares each block's hash against a recalculated one and the current block's stored previous hash against the previous block's hash. This allows the whole chain to validate and pinpoint exactly which block was altered. Because blocks also have a block ID, if the block IDs are discovered to be non-sequential, it can be shown that a block was deleted.

The EvidenceLedger (Fig. 2) is a dictionary of blockchains, where each key is the evidence ID associated with a blockchain. The item in the key-value pair is the blockchain. The ledger validation begins by validating each blockchain independently. It then creates a list of individual blocks and sorts them by their global ID, creating a repeatable structure for building a state root hash. The ledger was crucial to enabling blockchains to be validated and accessed independently while preserving validation of the entire ledger. It also allows users to access their chain of custody records independently via evidence ID and signer ID.

The state root hash is built by combining the state root at each block addition with the hash of that block, then using SHA-256 to encode the resulting byte stream. If the recalculated state root hash doesn't match the stored state root hash, the ledger isn't valid. It also utilizes signature verification during validation, but that is to show knowledge of cryptographic signing and verification in place of using a certificate authority.

The signatures used in this application are based on the Elliptic Curve Digital Signature Algorithm (ECDSA). This was chosen because of its popularity in blockchains such as Bitcoin and Ethereum, as it provides strong security while utilizing small keys to maintain efficiency.

Two features I included in the backend but did not utilize in the GUI, as the simulation only acts as a single node, are the validation checkpoint and checkpoint verification. When multiple nodes are on the network, they can create a signed checkpoint validation that includes the validator's ID (node), the ledger state root hash, the timestamp, and the validator's cryptographic signature. That checkpoint is then sent to another node to be verified. If their state root hashes don't match or the signature isn't verified, the checkpoint verification fails. This serves as a simulation of a consensus mechanism for nodes on the network to maintain ledger integrity.

Some limitations of this application are that it is not implemented as a functional distributed network. I also couldn't implement the verification checkpoint features because I am using the timestamp in the block hash function. The timestamp is important for chain of custody integrity, and creating two separate ledgers independently created differences in the timestamps, causing the validations to fail even if the blocks were otherwise identical.

Another limitation is the simulation of cryptographic signing. The keys are stored on the block for convenience, but it is not how I would implement it on a production application. In reality, the private key should never be stored on the blockchain and should remain securely stored on a user's system. The private key would be used to sign the hashes for each block, ledger, or checkpoint validation. Public keys should be retrieved from a public key infrastructure (PKI) and used for signature verification.

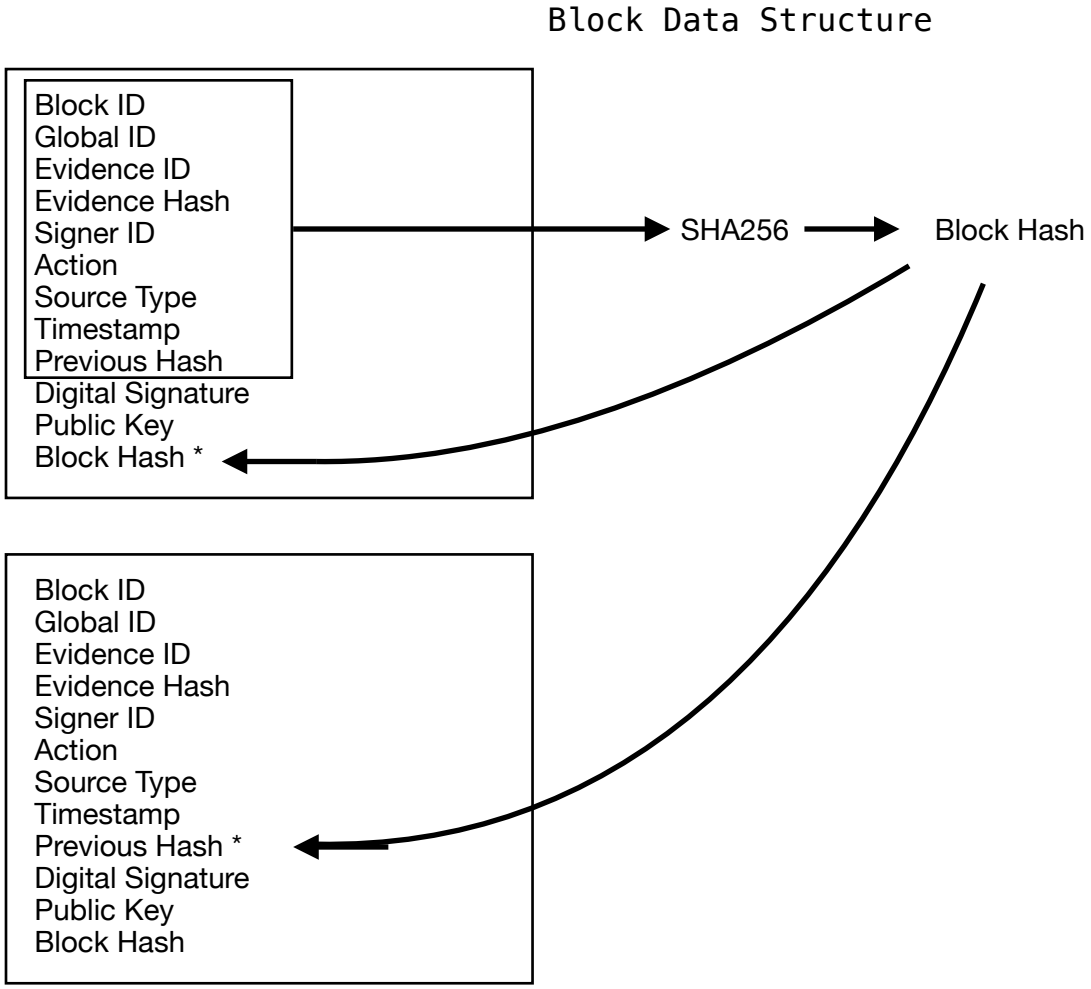

Figure 1.

Figure 1 shows a block and its contents. The inner box is the data used to generate the block hash. The block hash is generated and stored on the block and referenced as the previous hash in the next block on the chain.

EvidenceLedger Data Structure

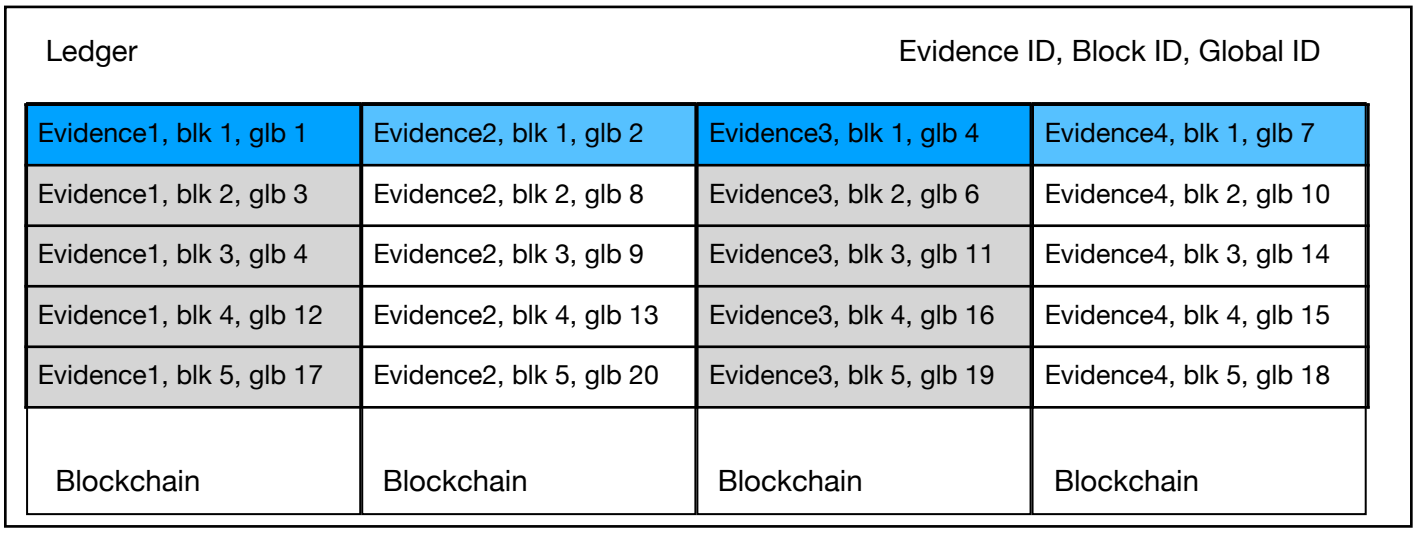

Figure 2.

As shown in Figure 2, each blockchain represents the chain of custody for a single evidence item identified by a unique evidence ID. Within each blockchain, block IDs are assigned sequentially, while Global IDs are assigned across the entire ledger as new blocks are added. Because Global IDs aren't sequential within an individual blockchain, the ledger must construct and sort a list of all blocks by Global ID before calculating the cumulative state root hash.

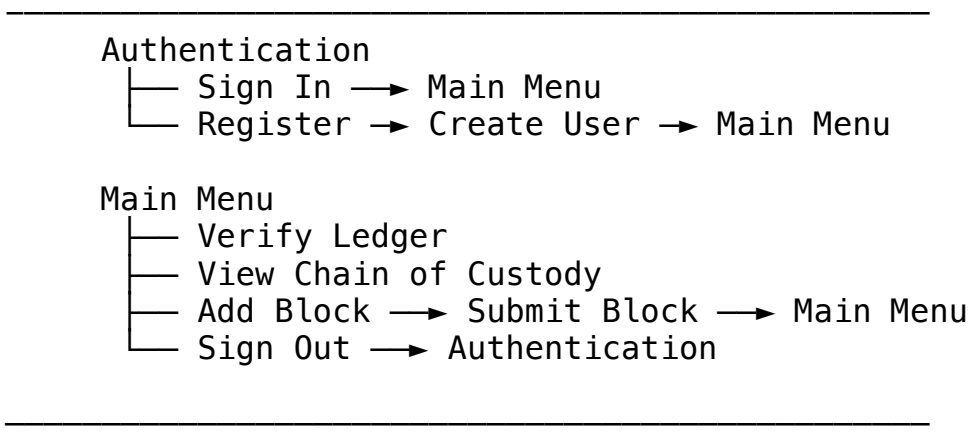

Figure 3.

Figure 3 is a simple user interaction flow for menu options.

Conclusion

This application serves as a proof of concept for how blockchain technology can be utilized with digital forensic software and evidence handling to create a digital chain of custody record. This technology allows for integrity validation checks, non-repudiation of record submission, and, theoretically, distributed authorized access to the chain of custody. It features cryptographic hashing, ECDSA signatures, the ability to create and verify blocks, and sanitization of input as a front-end defense to block submission.

Project architecture was inspired by the following projects and papers

Open Source:

https://github.com/darshchaurasia/Chain-Of-Custody-Blockchain-Tool/blob/main/blockchain.py
Used as research into Python-based blockchain development.

https://github.com/debugmaster0/little-life_microscope
Personal projects used it as a reference for implementing UI in this project.

Research References:

https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8387.pdf?utm_source=chatgpt.com

Yang, L., Jiang, R., Pu, X. et al. An access control model based on blockchain master-sidechain collaboration. Cluster Comput 27, 477–497 (2024). https://doi-org.unr.idm.oclc.org/10.1007/s10586-022-03964-x

Ahmad, L., Khanji, S., Iqbal, F., & Kamoun, F. (2020, August 25). Blockchain-based chain of custody: towards real-time tamper-proof evidence management. Proceedings of the 15th International Conference on Availability, Reliability and Security, Article 48. https://doi.org/10.1145/3407023.3409199

Haji, I. A., Mohammed, S. D., & Mousa, K. M. (2026). Blockchain based chain of custody and digital evidence legality in post conflict prosecutions. Frontiers in Blockchain, 9, Article 1801364. https://doi.org/10.3389/fbloc.2026.1801364

V. A. Pasenchuk and T. N. Demchenko, "Hybrid Cryptographic Architecture for Multi-Platform Blockchain Document Signing: Technical Innovation and Cross-Chain Interoperability," *2025 Dynamics of Systems, Mechanisms and Machines (Dynamics)*, Omsk, Russian Federation, 2025, pp. 1-6, doi: 10.1109/Dynamics68764.2025.11302162.

Acknowledgments

ChatGPT was used to assist with PySide6 format issues and general debugging. Block, blockchain, ledger architecture, signing utilities, and code were implemented by Bruce Martin.